بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

In the name of Allah, Most Gracious, Most Merciful

CSE 4107
Structured Programming I

Topic: Selection Statements

Asaduzzaman Herok
Lecturer | CSE | IUT
asaduzzaman34@iut-dhaka.edu

Selection Statements

Three categories based on how they affect the order in which they are executed.

Follow the white board

- *Selection statements*: The `if` and `switch` statements (this lecture)
- *Iteration statements*: The `while, do while, for` statements (looping next lecture)
- *Jump statements*: The `break, continue` and `goto` statements. (`return` statement also belongs to it, next lecture)

Some other statements:

- *Compound statements:* groups several statements into one. (this lecture)
- *Null statements*: performs no action. (next lecture)

Source: C Programming Modern Approach Book

Logical Expressions

Testing an expression value to see if it is `true` or `false.`

`Follow the white board`

- If statement needs to check expressions: Example `if(i < j)`
- Comparison in C: `( i < j )` yields an integer either `0` (meant as false) or `1` (meant as true)
- Other programming language: have special type Boolean or Logical.

Source: C Programming Modern Approach Book

Logical Expressions

Relational Operators: `<, > <=, >=` produces `0` (false) or `1` (true). Left associative.

Follow the white board

```
10 < 11       //Represents 1
10 > 11       //Represents 0
```

- Compares integer and floating-point numbers with operands of mixed types allowed.
- Precedence is lower than the arithmetic operators. `i+k < k-1` is equivalent to `(i+k) < (k-1)`

Symbol	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

Bonus question ?

`i < j < k`

Source: C Programming Modern Approach Book

Logical Expressions

Equality Operators: `==, !=` produces `0` (false) or `1` (true). Left associative.

Follow the white board

```
10 == 10      //Represents 1
10 != 11      //Represents 0
```

- Compares integer and floating-point numbers with operands of mixed types allowed.
- Precedence is lower than the relational operators. `i<k ==j<k` is equivalent to `(i<k) == (j<k)`
- Problem comparing equality of two floating point numbers.

Symbol	Meaning
==	Equal to
!=	Not equal to

Source: C Programming Modern Approach Book

Logical Expressions

Logical Operators: `!` (unary, right associative), `&&`, `||` (binary left associative). produces `0` (false) or `1` (true).

Follow the white board

```
!( 10 > 11 )        //Represents 1
(10 < 11) && (11 < 12)        //Represents 1
```

- Precedence is lower than the relational and equality operators. `i<k && j==k` is equivalent to `(i<k)&& (j==k)`
- `!expr` has value `1` if expr has value `0`.
- $expr_1$ `&&` $expr_2$ has value `1` if both of $expr_1$ and $expr_2$ are non-zero.
- $expr_1$ `||` $expr_2$ has value `1` if at least one of $expr_1$ and $expr_2$ is non-zero. Lower precedence than `&&`

Symbol	Meaning
!	Logical negation
&&	Logical and
||	Logical or

Short-circuit evaluation

Source: C Programming Modern Approach Book

The `if` Statement

`if` Statement: allows to choose between two alternatives.

Follow the white board

```
if ( expression ) statement
```

- Evaluate the expression: is non-zero ? execute the statement.
- Common mistake on equality check `if ( var = value).`

Source: C Programming Modern Approach Book

The `if` Statement

Compound Statement: set of statements inside a curly braces {}.

Follow the white board

```
{
    statement
    statement
    … … … …
}
```

Compound Statement

```
if ( expression ) {
    statement
    statement
    … … … …
}
```

Compound `if` Statement

- Evaluate the expression: is non-zero ? execute the compound statement.

Source: C Programming Modern Approach Book

The `if` Statement

The `else` Clause: statement/compound statement after else executed when expression in evaluated to zero (the false).

Follow the white board

```
if ( expression ) statement else statement
```

```
if ( expression ) {
     statement
     … … … …
} else {
     statement
     … … … …
}
```

Source: C Programming Modern Approach Book

The `if` Statement

Cascade `if` statement: series of conditions, stops as soon as one of them is true.

Follow the white board

```
if ( expression1 ) {
     statement
     … … … …
} else if ( expression2 ) {
     statement
     … … … …
} else if ( expression3 ) {
     statement
     … … … …
} else  {
     statement
     … … … …
}
```

Source: C Programming Modern Approach Book

The `if` Statement

Dangling `else` problem:

Follow the white board

```
if ( y != 0 )
     if ( x != 0 )
          result = x / y;
else
     printf("Error: y is equal to 0\n"
```

```
if ( y != 0 ) {
     if ( x != 0 )
          result = x / y;
} else
     printf("Error: y is equal to 0\n"
```

Source: C Programming Modern Approach Book

The `if` Statement

Conditional Expression: an operator that allows an expression to produce one of two *values* based on the value of a condition. (ternary operator)

Follow the white board

$$expr_1 \ ? \ expr_2 \ : \ expr_3$$

- Read as if $expr_1$ then $expr_2$ else $expr_3$
- Evaluate first $expr_1$; if its value non-zero then $expr_2$ and its value is the value of the entire expression.
- If $expr_1$ evaluates to zero then evaluate $expr_3$ and its value is the value of the entire expression.

```
i = 1, j = 2;
k = i > j ? i : j;                  // k is now 2
k = ( i >= 0 ? i : 0) + j           // k is now 3
```

Source: C Programming Modern Approach Book

The `switch` Statement

Switching: compare an expression value against a series of values to see which one it currently matches. Alternative to cascaded `if` statement

Follow the white board

```
switch ( expression ) {
     case constant-expression : statements
     … …
     case constant-expression : statements

     default : statements
}
```

- Easier to read than cascaded `if` statements.
- Faster than `if` statements when there are more than a handful of cases. Optimized during compilation of code.

Source: C Programming Modern Approach Book

The `switch` Statement

Components: Controlling expression, Case labels, Statements.

Follow the white board

- Controlling expression: the expression in the parentheses.
  - Expression value can't be floating point numbers nor strings.
  - Must be evaluated to an integer value.
  - Character can be used as treated as integer.
- Case Labels: begins with a label of the form

  case *constant-expression* :

  An ordinary expression and can't contain variables or function calls.

```
case    5     :    // Okay
case    5+10  :    // Okay
case    5+n   :    // Wrong
```

  Must evaluate to an integer.

Source: C Programming Modern Approach Book

The `switch` Statement

Components: Controlling expression, Case labels, Statements.

Follow the white board

- Statements: can have any number of statements.
  - No curly braces.
  - Last statement in each group of statements is normally `break`
- Duplicate case not allowed
- Order of case doesn't matter.
- Only one constant expression may follow the word `case`
- Several case labels may precede the same group of statements
- No way to specify range of values.
- Not required to have `default` case

Source: C Programming Modern Approach Book

The `switch` Statement

The role of `break` statement: causes to break out of the switch statement.

Follow the white board

- Switch statement is really a form of "computed jump".
- After evaluating the controlling expression, control of the program jumps to the case label.
- Case label is nothing but a marker indicating a position.
- Control falls through the next case statements without break.

```
switch ( grade ) {
     case 4: printf( "Excellent" );
     case 3: printf( "Good" );
     case 2: printf( "Average" );
     case 1: printf( "Poor" );
     case 0: printf( "Failing" );
     default : printf( "Illegal grade" );
}
```

Source: C Programming Modern Approach Book


Q
A


THANK
YOU